PYTHON FOUNDATIONS - ASSIGNMENT 08

Strings

Indexing, Slicing & Real-World Text Handling

Scenario-based Python challenges covering indexing, slicing defaults, step, reverse slicing, loops, membership, validation and essential string methods.

indexing | slicing | len() | loops | methods | validation

Name: _____

Date: _____

Batch: _____

BEGINNER TRACK

14 SCENARIO CHALLENGES

Quick reference before you begin

Remember: slicing follows text[start : stop : step].

Start is included, stop is excluded, and omitted values use Python's natural defaults.

Indexing

text[0] -> first character

text[-1] -> last character

Slicing

text[:4] -> beginning to index 3

text[4:] -> index 4 to end

Step

text[::2] -> every 2nd char

text[::-1] -> reverse

Methods you may use

Case

lower(), upper(), title(), capitalize()

Clean

strip(), replace()

Search

find(), count(), in / not in

Checks

isdigit(), isalpha(), isalnum(), isspace()

Edges

startswith(), endswith()

Break apart

split()

Use only the concepts already covered. Do not use functions unless your teacher asks you to.

Write clean output labels so a tester can understand what your program is reporting.

PART A - INDEXING & SLICING

1 Student Badge Inspector

indexing | negative index | len()

SCENARIO

A training portal stores a student badge text such as "PY-2026-VINEET". Build a small inspector that reports important parts of the badge.

Your program must:

a. Take the badge text from the user and print its total number of characters using len().
b. Print the first character, last character, second character and second-last character using indexing.
c. Print the first two characters as the course prefix using slicing.
d. Print the last six characters using negative slicing.
e. Print whether the first and last characters are the same.

2 Delivery Tracking Code Slicer

slicing defaults | start | stop

SCENARIO

A courier system receives a tracking code such as "HYD2026EXPRESS". The support team wants different parts of the same code without manually rewriting it.

Your program must:

a. Print the first three characters using a slice with an omitted start value.
b. Print everything from index 3 to the end using an omitted stop value.
c. Print the complete code using [:].
d. Print characters from index 3 up to, but not including, index 7.
e. For each slice above, write a comment identifying the effective start, stop and step.

PART A - INDEXING & SLICING

3 Security Token Sampler

step | slicing | reverse

SCENARIO

A test environment produces a token string such as "A1B2C3D4E5F6". QA wants to sample characters at regular intervals.

Your program must:

a. Print every second character starting from the beginning using [::2].
b. Print every second character starting from index 1 using [1::2].
c. Print the token from index 2 to index 10 using a step of 2.
d. Print the token in reverse using [::-1].
e. Print the reversed token while taking every second character.

4 Mirror Word Challenge

[::-1] | if / else | lower()

SCENARIO

A puzzle application needs to detect words that look the same from left to right and right to left.

Your program must:

a. Take a word from the user and normalize it to lowercase.
b. Create a reversed version using slicing only.
c. Print both the original normalized word and reversed word.
d. Use if/else to print "Palindrome" when both are equal, otherwise print "Not Palindrome".
e. Also print the first half of the word using slicing.

PART B - STRING METHODS & VALIDATION

5

Customer Name Cleaner

strip() | case methods | replace()

SCENARIO

A registration form often receives names with extra spaces and inconsistent capitalization, for example " vInEeT tiWaRi ".

Your program must:

a. Remove spaces from the beginning and end.
b. Print the cleaned name in lowercase, uppercase and title case.
c. Print only the first character of the cleaned name in uppercase.
d. Print the length before cleaning and after cleaning.
e. Check whether the cleaned name contains only alphabetic characters after removing the single space between first and last name using replace().

6

Email Address Analyzer

membership | find() | endswith()

SCENARIO

A test script needs to quickly inspect an email entered by a user.

Your program must:

a. Remove outer spaces and convert the email to lowercase.
b. Check that the email contains both "@" and "." using membership operators.
c. Find the position of "@" using find().
d. Use slicing to print the username portion before "@".
e. Check whether the email ends with "gmail.com" and count how many times "@" appears.

PART B - STRING METHODS & VALIDATION

7

File Upload Gatekeeper

endswith() replace() membership

SCENARIO

An automation test must validate whether a selected file is an allowed document before upload.

Your program must:

a. Take a file name and normalize it to lowercase.
b. Accept the file only if it ends with .pdf, .docx or .xlsx.
c. Print whether the name contains the word "report".
d. Replace spaces in the file name with underscores and print the cleaned file name.
e. Print the last four characters of the cleaned file name using slicing.

PART C - LOOPS, SEARCH & REAL-WORLD TEXT

8

Support Message Character Report

for loop | isalpha() | isdigit()

SCENARIO
A support chatbot receives a message and the QA team wants a character-level report.

Your program must:

a. Loop through every character in the message.
b. Count vowels and consonants separately.
c. Count digits and spaces separately.
d. Count uppercase and lowercase letters separately.
e. Print all six counts with clear labels. Ignore symbols when counting consonants.

9

Incident Keyword Scanner

count() | find() | replace()

SCENARIO
An incident note may contain repeated words such as "error" and an old service name that needs to be replaced.

Your program must:

a. Normalize the note to lowercase for case-insensitive checking.
b. Check whether the word "error" exists.
c. Count how many times "error" appears.
d. Find the position of the first occurrence of "error".
e. Replace every occurrence of "oldservice" with "newservice" and print the updated note.

PART C - LOOPS, SEARCH & REAL-WORLD TEXT

10

Secure URL Checker

startswith() endswith() strip()

SCENARIO

A browser automation suite receives URLs from test data and should reject obviously invalid endpoints.

Your program must:

a. Remove extra outer spaces from the URL.
b. Check whether it starts with "https://".
c. Check whether it contains the word "login".
d. Check whether it ends with "/login" or "/signin".
e. Print the URL in lowercase and print its total length.

11

Username Quality Check

isalnum() len() validation

SCENARIO

A website requires usernames that are easy to store and validate.

Your program must:

a. Take a username, remove outer spaces and convert it to lowercase.
b. Check that its length is at least 5 characters.
c. Check that it is not one of these reserved values: admin, root, system.
d. Use isalnum() to check whether it contains only letters and digits.
e. Print a separate failure reason for every rule that the username violates; otherwise print "Username Accepted".

PART C - LOOPS, SEARCH & REAL-WORLD TEXT

12

Password Strength Inspector

loop | character checks | validation

SCENARIO

A registration screen needs a basic password-strength check. Do not use regular expressions.

Your program must:

a. Take a password and verify that its length is at least 8.
b. Loop through the password and detect whether at least one uppercase letter exists.
c. Detect whether at least one lowercase letter and at least one digit exist.
d. Detect whether at least one of these special characters exists: @, #, $, !.
e. Print "Strong Password" only when every rule passes; otherwise print which requirements are missing.

PART D - FINAL CHALLENGES

13

CSV Profile Reader

split() indexing membership

SCENARIO

A small test-data line arrives as "Vineet,Hyderabad,Python,Senior". You need to break it into meaningful values.

Your program must:

a. Take the comma-separated profile string from the user.
b. Use split(",") to separate the values.
c. Print the first item as Name and the second item as City.
d. Print the third item as Skill and the fourth item as Level.
e. Print how many characters are present in the original profile string and whether the word "Python" exists in it.

14

Final Registration Text Validator

final challenge combined concepts real-world

SCENARIO

Build a single console program for a registration screen. It should validate full name, username, email and password using the string concepts from this assignment.

Your program must:

a. Full name: strip outer spaces, convert to title case, and ensure it is not blank or only spaces.
b. Username: lowercase it, require at least 5 characters, reject admin/root/system, and require letters/digits only.
c. Email: lowercase it, require @ and ., print the username part before @, and check whether it ends with gmail.com.
d. Password: require at least 8 characters and detect uppercase, lowercase, digit and one special character from @ # $! using a loop.
e. Print a clear summary showing the normalized values and whether each field passed or failed.

Submission checklist

Before you submit, check these five things:

1. **Your program runs without an error for normal input.**
2. **Your output labels make it clear what each result means.**
3. **You used slicing correctly: start included, stop excluded.**
4. **You normalized user input where case or extra spaces could cause a false failure.**
5. **You did not copy a built-in shortcut when the question specifically asks you to practise a loop.**

Stretch goal

Choose any two scenarios and make the program case-insensitive wherever appropriate. Then test each scenario using at least three different inputs, including one invalid input.